Lecture 14

Name	Roll Number
Karthikeya Hanu Prakash Kanithi	EE22BTECH11026
Pandrangi Aditya Sriram	EE22BTECH11039
Rambha Satvik	EE22BTECH11043
Gullapalli Madhava Asrith Murthy	AI23BTECH11007

12 September, 2024

1 Control Hazards

1.1 Introduction

- The delay in determining the proper instruction to fetch, when executing a branch instruction, is called a control hazard or branch hazard
- A branch that changes the PC to its target address is called a "taken" branch, while if it falls through (i.e. current value plus 4.), it is "not taken."
- The next instruction to be fetched depends on the branch outcome (taken or not taken). The pipeline can only determine the next instruction to fetch after MEM stage,
- The simplest (inefficient) method to address this hazard is to **stall on branch**, i.e. Wait until branch outcome is determined before fetching next instruction
- In case the branch outcome is determined in MEM stage as "taken", the instructions already in the pipeline are flushed by setting the control values to zero. No extra care is necessary, as the Data Memory or Register Writeback stage for any of the following flushed instructions are not executed yet in RISC-V pipeline (this may not be the case for other pipelines).
- This causes a waste of 3 cycles, as the decision takes place in the 4th stage.
- In case the branch outcome is "not taken", then the execution may continue as normal with no wastage of cycles.

1.2 Modifying the hardware

- One approach to reduce the number of stall cycles is to be able to determine branch outcome earlier, rather than waiting till the MEM stage
- Additional hardware is added in the decode (ID) stage to compare the registers as well as calculate the target address in case of branch taken, without waiting till MEM stage.

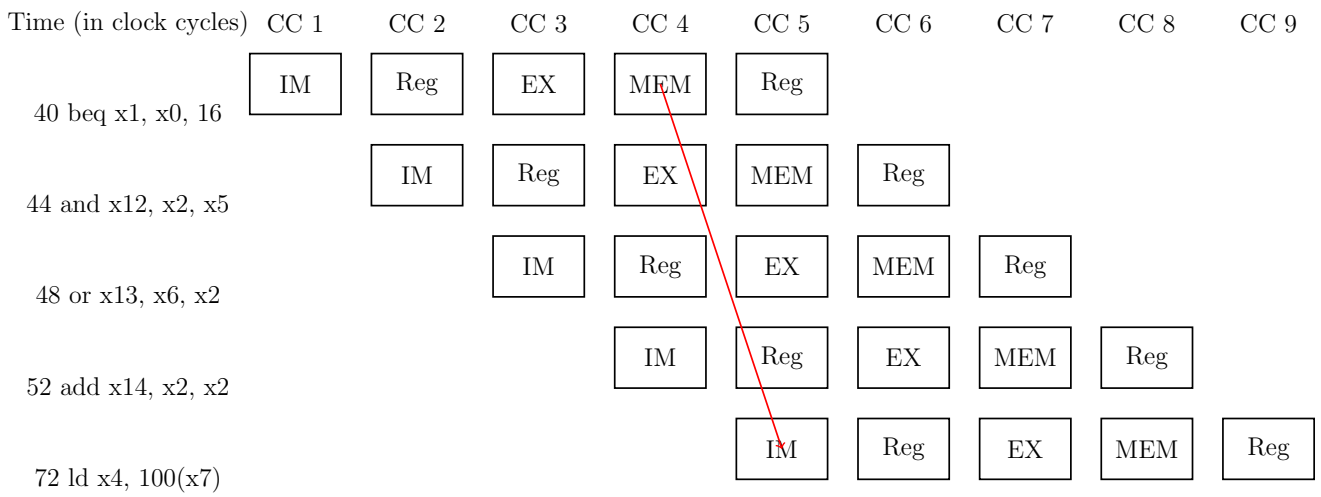


Figure 1: Branch outcome determined as taken in MEM stage, flushing middle three instructions.

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX

Figure 2: In case of a simple one-cycle stall on branch in the five-stage pipeline. The instruction after the branch is fetched, but is ignored, and the fetch is restarted once the branch target is known. It is obvious that if the branch is not taken, the second IF for branch successor is redundant.

- Moving ALU for register comparison to the second stage (ID) may not be feasible as it drastically increases the delay in this stage; potentially increasing clock cycle.
- Instead, we add an equality checker (consisting of XOR/XNOR gates only) and move the adder to the 2nd stage or ID stage.
- The equality checker will evaluate the branch decision 2 stages earlier and we can use that decision signal to change the PC to branch target using the adder in the same stage.
- We also move the adder to calculate the branch target address (PC+offset) to the ID stage.
- This reduces the stall to 1 cycle. The first Instruction Fetch (IF) cycle after a branch is essentially a stall since the outcome is not known yet.
- If the branch is not taken, the repetition of the IF stage does not happen and the flow continues as normal. It is unnecessary because the correct instruction was fetched initially.
- However, in this optimization, moving the branch test to the ID stage implies additional forwarding and hazard detection hardware, since a branch dependent on a result still in the pipeline must still work properly.

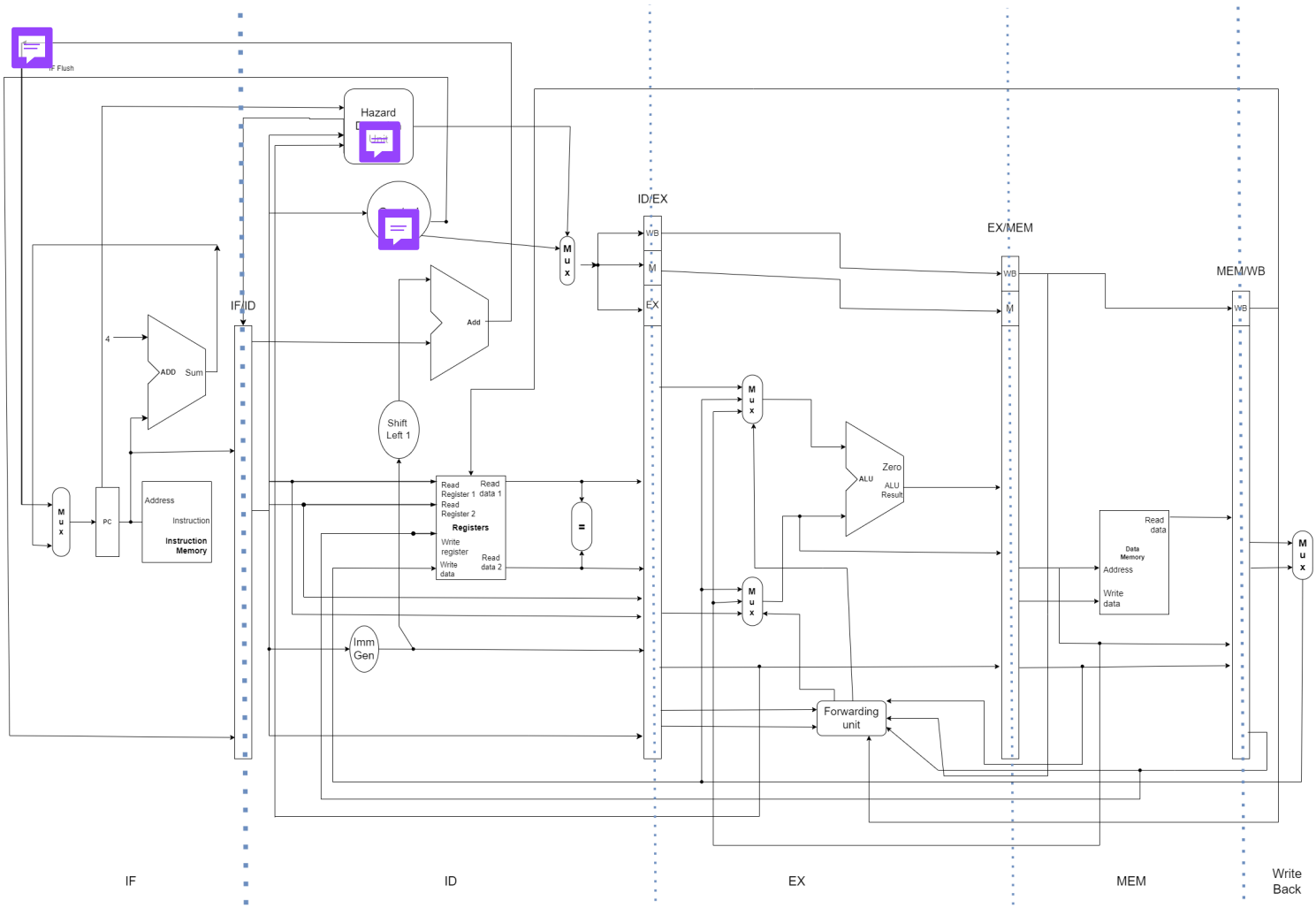


Figure 3: Datapath with changes in the hardware to accommodate the branch test and adder-to-PC in the 2nd stage. Please download the image "Final-datapath.png" for higher resolution.

- For example, if a load is immediately followed by a conditional branch that depends on the load result, some stall cycles will be needed, as the result from the load appears at the end of the MEM stage but is needed at the beginning of ID for the branch.

1.3 Branch Prediction

- Longer pipelines can't readily determine branch outcome early, since fetch instructions might take more than 1 stage. Stall penalties may become unacceptable.
- Therefore, we predict whether the branch will be "taken" or "not taken".
- Stalls take place only when the prediction is wrong.
- This follows the paradigm of "make the common case fast".
- Types of branch prediction:

1. Static prediction
   - Based on typical branch behaviour.
   - Predict backward branches taken.
   - Predict forward branches not taken.
   - For example, the loop, a backward branch, is usually taken. A loop counting to 1000 would be predicted correctly in 999 cases, and incorrectly only in the exit case.
2. Dynamic Prediction
   - Prediction of branches at runtime using runtime information.
   - Hardware keeps track of recent behaviour of each branch.
   - Assume future behaviour will continue the trend from past.

1.3.1 Dynamic Branch Prediction

- A branch prediction buffer (a branch history table) is created in hardware.
- This is indexed by the recent branch instruction addresses, and stores the outcome of that particular branch (taken/not taken).
- Whenever a branch is encountered, we check the table, and expect the same outcome. Then we start fetching from either the next instruction (fall-through) or the branch target. If the prediction is wrong, flush the pipeline and flip prediction.
- A more complex predictor would give higher accuracy in the prediction at the cost of more hardware.

1.3.2 Dynamic Branch Prediction using 1-Bit Predictor

- This predictor only stores the most recent outcome for each branch instruction address.
- **Shortcoming:** Using this predictor, the inner loop branches are mispredicted twice. Consider the code:

```
outer:  ...
        ...
inner:  ...
        ...
        beq ..., ..., inner
        ...
        beq ..., ..., outer
```

- Here, the inner loop is always mispredicted as taken on last iteration. Also, the first iteration of the inner loop in the next iteration of the outer loop is always mispredicted as not taken. Thus, two mispredictions occur in every cycle of the outer loop. This is a very common occurance in programming.
- To reduce the number of mispredictions, we use a 2-Bit predictor instead.

1.3.3 Dynamic Branch Prediction using 2-Bit Predictor

- The prediction is changed only on two successive wrong predictions. In the above example, there would now be only one misprediction in every cycle of the outer loop.
- This is described like a Finite State Machine (FSM). An FSM consists of some states (here, predict taken, predict not taken, etc) with arrows representing the next state in case of a certain condition "input" (here, taken or not taken).

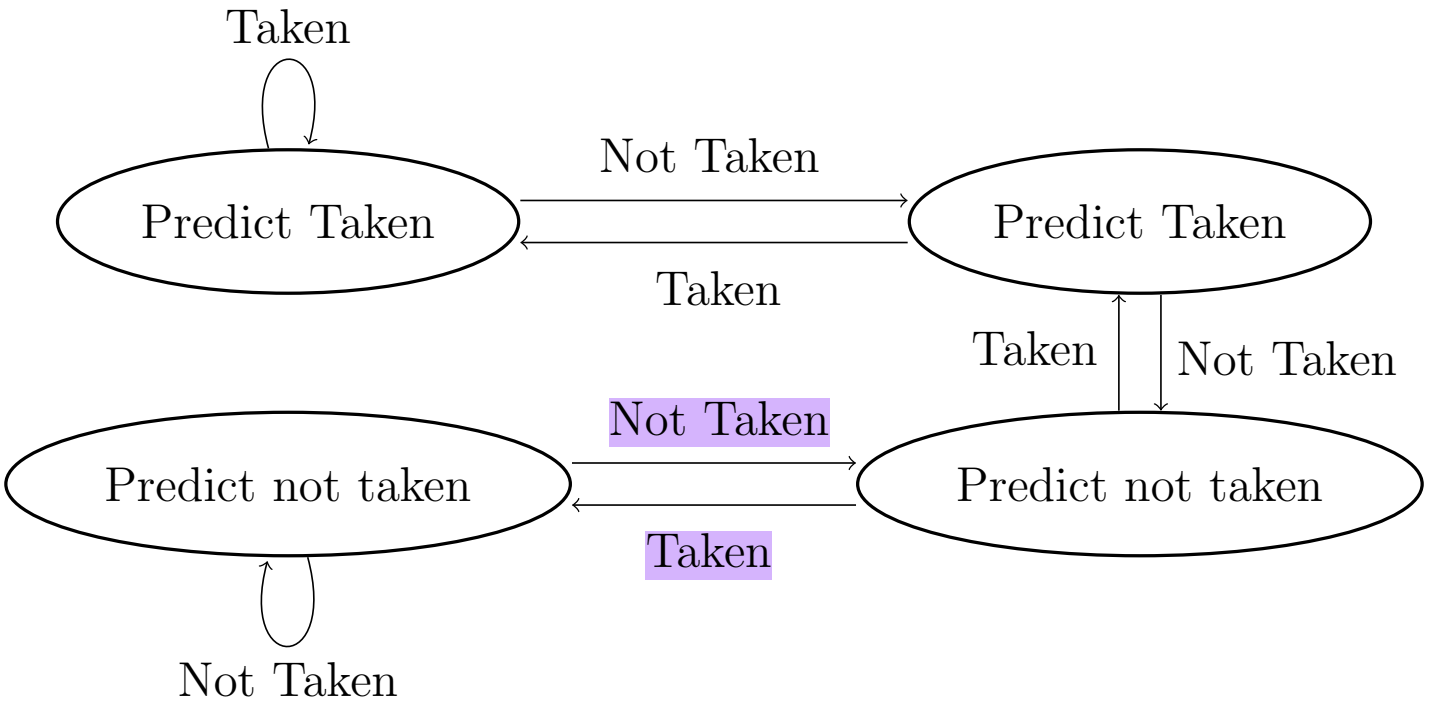


Figure 4: The states in 2-bit prediction schema.